CS3081 CLOUD COMPUTING MINI PROJECT

Knowledge Distillation and Deployment of a Lightweight Command-Parsing Language Model on a Local Cloud

Project Report

Student	Course / Environment	Submission date
Sambodh Gupta | Roll No. 230101019	CS3081 Cloud Computing | k3d on a local Ubuntu/WSL laptop	9 November 2026

Executive summary

This project builds an end-to-end command-parsing pipeline: a larger teacher model generates structured command labels; a compact Pythia-70M student is trained on those examples; the model is stored in S3-compatible object storage; and a FastAPI service loads it into a Kubernetes deployment. The work also covers subnet design, network access-control simulation, CI quality gates, load testing, recovery, and cost estimation.

Measured headline results: 3,000 teacher-labelled candidates were cleaned to 2,864 records; v2 achieved 98% JSON validity and 37% full-frame exact match on the recorded validation set; the corrected API returned 49/50 HTTP 200 responses in the latest sequential test (p95 0.977 s); Pod recovery under load had 1.11% failures; cold-start median was 12 s across six Pods. Rolling-update failure rate was 3.19%, above the 0.5% requirement.

System architecture

Data and labelling: command pool → two teacher-labelling shards → cleaner → SeaweedFS *datasets* bucket. **Training:** cleaned JSONL → Pythia-70M fine-tuning → v1/v2 model archives → SeaweedFS *models* bucket and GitHub Release. **Serving:** FastAPI container → Kubernetes Deployment / Service / Ingress / HPA → /parse API. **Assurance:** pytest, Ruff, GitHub Actions, Gitleaks, Locust, and NACL simulator.

Scope and evidence convention

Numbers in this report are identified as measured, derived, or assumed. Measurements come from the project logs, CSV/JSON result files, Kubernetes output, and Actions runs. Cost uses the supplied hypothetical rate card; it is an estimate, not a cloud invoice. The local k3d cluster runs on one laptop, so its nodes do not provide physical multi-machine high availability.

Task 1 — Object storage and parallel teacher labelling

The data pipeline starts with a pool of command examples. A Phi-3.5 teacher was used to produce structured command frames. The workload was split into two shards, each containing 1,500 examples, and executed as two Kubernetes labelling jobs.

Measured data flow

Stage	Measured result	Evidence / note
Command pool / teacher output	3,000 candidate records	Two shards; 1,500 records each
Shard 0 SHA-256	40096c347ba0c98b1ad9a8b3e2c79a21aa01183a6ff11e5548126f8abc33f4f5	Integrity digest recorded
Shard 1 SHA-256	39301e2173ea3928f76ecb094487e5691a03f288611a46ae2bd0688aebc8efc4	Integrity digest recorded
Cleaning	2,864 kept; 136 removed	48 duplicates + 88 bad-slot records
Clean dataset SHA-256	08e51e29a7c62150180da169eb68a6fdb20ddd7ae8930d2aa67bcb272105f592	datasets/v1/train.jsonl
Kubernetes jobs	2 completed; 1,500 records each; 0 failed	label-shard-0 and label-shard-1
Object storage	7 dataset objects; 16 model objects	SHA verification performed for model files

Cleaning summary

The cleaner retained 2,864 of 3,000 candidates. The recorded removal categories are 48 duplicate records and 88 records with invalid slot content. The detailed per-category counts for bad JSON, bad keys, and invalid intents are not reproduced here because the evidence currently available for this report does not contain their individual counts; the aggregate removal count is consistent: 3,000 - 2,864 = 136.

Sharding and object-storage design

Two independent shards allow teacher labelling to run in parallel rather than processing the complete input in one worker. Each job writes its shard output separately, which supports debugging and integrity checks before cleaning. SeaweedFS provides an S3-compatible object interface inside the local cluster. The project uses separate **datasets** and **models** buckets to distinguish source/training data from deployable model artifacts.

Reproducibility note: the cleaned dataset is stored as datasets/v1/train.jsonl. The SHA-256 value above identifies the exact cleaned file used for training.

Limitations

The storage and job execution were validated on a local k3d cluster. The 7 dataset and 16 model object counts describe the observed bucket contents in the project run; they are not a claim of a production object-store durability or availability guarantee.

Task 2 — Knowledge distillation and student training

A Phi-3.5 teacher produced structured JSON target frames. The student was Pythia-70M, fine-tuned on 1,500 cleaned examples. This implementation uses sequence-level supervision: the teacher-generated output sequences are used as targets for ordinary token-level training loss. It is not logit-based distillation because teacher probability distributions/logits were not transferred to the student.

Training runs

Run	Epochs	Training duration	Final training loss	Recorded epoch losses
v1	3	2,828.59 s	0.8111	1.1311 → 0.7343 → 0.5680
v2	4	3,652.72 s	0.7449	1.1418 → 0.7610 → 0.6023 → 0.4746

Validation comparison

Model	JSON validity	Intent	Date	Time	Place	Full frame
Teacher	100%	83%	77%	81%	78%	39%
Untrained Pythia-70M	0%	0%	0%	0%	0%	0%
Student v1	99%	79%	71%	80%	79%	33%
Student v2	98%	81%	73%	80%	81%	37%

Interpretation

The four-epoch v2 run reduced training loss relative to v1 and improved several validation components: intent rose from 79% to 81%, date from 71% to 73%, place from 79% to 81%, and full-frame exact match from 33% to 37%. JSON validity was 98%, one percentage point below v1. The teacher remained stronger overall, particularly on full-frame exact match (39%). Therefore the student is substantially smaller and deployable, but it does not fully reproduce teacher quality.

Why sequence-level rather than logit distillation?

Sequence-level distillation is practical when the teacher is accessible as a text-generation API or local inference endpoint but does not expose compatible logits. The teacher converts natural-language commands into target structured sequences. The student learns to generate those sequences from the command prompt using supervised fine-tuning. Logit-based distillation would instead require teacher distributions at each token position and a loss comparing teacher and student distributions; that was not part of this pipeline.

Artifact evidence: v1 and v2 model files were uploaded to object storage and released as downloadable archives with SHA256SUMS. Each compressed release archive was approximately 216.18 MiB.

Task 3 — Containerised inference service

The FastAPI service loads tokenizer and model artifacts from the S3-compatible object store at startup. It exposes /parse for command parsing, /healthz for process liveness, and /readyz for model readiness. The service validates the parsed output against data/schema.json and returns the required error object when generated output is malformed or fails the schema.

Service contract

Endpoint / control	Observed or implemented behaviour
/healthz	HTTP 200 liveness response; checked directly through a Pod port-forward
/readyz	HTTP 503 until model loading completes, then HTTP 200; final direct check returned 200
/parse	HTTP 200 for valid command output; response includes X-Model-Version and X-Served-By
Invalid output	HTTP 500 with exact body {"error":"invalid_model_output"}
Fault injection	FAULT_INJECT=1 forces invalid text and exercises schema/error path
Environment	MODEL_URI, MODEL_VERSION, S3_ENDPOINT, MAX_NEW_TOKENS; S3 keys supplied via Kubernetes Secret
Concurrency / CPU	One PyTorch intra/inter-op thread and an inference lock limit concurrent CPU inference

Container and dependencies

The service image was measured at 368,455,975 bytes (approximately 351 MiB) and does not bundle the model weights. Dependencies are pinned in service/requirements.txt: FastAPI 0.111.0, Uvicorn 0.30.1, Pydantic 2.7.4, boto3 1.34.131, PyTorch 2.3.1 CPU, and Transformers 4.42.3. The model is downloaded separately from object storage at startup.

Latest sequential API test

Metric	Measured result
Requests	50 sequential
HTTP 200 / failures	49 / 1 (98% HTTP 200 rate)
Mean / p50 latency	0.525 s / 0.523 s
p95 / maximum latency	0.977 s / 1.128 s
Failed input	V039: “i am no longer going to wyatts birthday party”
Failure response	HTTP 500: {"error":"invalid_model_output"}

The V039 output was observed to contain an incorrect intent and a repeated-character string that ended before a valid JSON frame was complete. Increasing the generation allowance to 128 tokens did not correct this example. The validation check was deliberately retained rather than accepting malformed output. The 49/50 result is the known-good decode-fix baseline; a later experimental repetition-penalty configuration was reverted.

Resource observation

After the 50-request run, kubectl top showed 650 MiB and 604 MiB for the two Pods. These are observed point-in-time memory readings, not a verified cgroup peak for the final image. Earlier pre-fix cgroup peaks were higher and are not used as the final-image peak claim.

Task 4 — Kubernetes deployment, scaling and reliability

The application runs in namespace app as a Deployment behind a ClusterIP Service and an Ingress route for /parse. The Deployment uses two replicas during load tests, resource requests of 250m CPU / 512 MiB and limits of 1 CPU / 1 GiB. Liveness uses /healthz and readiness uses /readyz. The HPA is configured for a minimum of one and maximum of two replicas with a 70% CPU target.

Rolling strategy and live update test

The deployment strategy was improved to maxSurge: 1 and maxUnavailable: 0 so a replacement can become ready before an old Pod is removed. A four-minute Locust run was active during the v1 → v2 update and rollback to v1. The ingress returned HTTP 200 with X-Model-Version v2 after the update and HTTP 200 with v1 after rollback.

Four-minute rolling test	Measured result
Total requests (Locust console)	596
Failures	19 (3.19%)
Failure breakdown	11 HTTP 502, 7 HTTP 500, 1 HTTP 0
p50 / p95 latency	1.3 s / 3.3 s
Average latency / throughput	1.598 s / 2.49 requests/s
Successful responses by version	v1: 481; v2: 96

Rubric gap: the required rolling-update failure rate is ≤0.5%. The measured 3.19% exceeds this target. The old strategy test was worse (5.64%); the improved strategy reduced failures, but did not meet the threshold.

Load balancing and Pod recovery

A separate two-minute test with two Ready replicas recorded 316 successful responses plus three failures; the two replicas served 158 successful requests each. During that run, observed peak CPU was approximately 1001m and 999m, and memory reached approximately 735 MiB and 701 MiB. Both Pods had zero restarts. A Pod deletion during a separate recovery test led to replacement and restoration of two Ready Pods. That test recorded 270 requests and three HTTP 500 failures (1.11%), below the 2% recovery threshold; successful v1 response headers were observed from three Pod identities during the replacement.

Cold-start results

Trial	Pod creation-to-Ready times	Trial median
1	12 s, 17 s	14.5 s
2	22 s, 12 s	17.0 s
3	12 s, 7 s	9.5 s
All six Pods	Overall median 12.0 s; mean 13.667 s	6 measurements

All six measured Pods logged both “Downloading model from” and “Model loaded from object storage and ready”. The k3d server and agent are both containers on one laptop. This validates Kubernetes behaviour in a local cluster but is not physical multi-machine high availability.

Task 5 — Subnetting and stateless NACL rules

The 10.20.0.0/16 address space was allocated largest subnet first using compact, non-overlapping CIDR blocks. The table below records the implemented ranges.

Subnet	CIDR	Usable host range	Broadcast	Usable hosts
Application	10.20.0.0/22	10.20.0.1 – 10.20.3.254	10.20.3.255	1,022
Public	10.20.4.0/24	10.20.4.1 – 10.20.4.254	10.20.4.255	254
Data	10.20.5.0/25	10.20.5.1 – 10.20.5.126	10.20.5.127	126
Management	10.20.5.128/27	10.20.5.129 – 10.20.5.158	10.20.5.159	30

Network ACL rule set

Direction / order	Protocol and port	Source / destination	Action
Inbound 100	All traffic	Source 198.51.100.0/24	DENY
Inbound 110	TCP 80	Anywhere	ALLOW
Inbound 120	TCP 443	Anywhere	ALLOW
Inbound 130	TCP 22	Management subnet	ALLOW
Inbound 140	TCP 1024–65535	Application subnet	ALLOW
Outbound 100	TCP 8000	Destination application subnet	ALLOW
Outbound 110	TCP 1024–65535	Anywhere	ALLOW
Default	All not matched	Any	DENY

The simulator evaluates rules in ascending order and stops at the first match. The explicit deny for 198.51.100.0/24 therefore takes precedence over the general web allows. Because a NACL is stateless, request and response directions must be permitted independently; the ephemeral-port rules allow return traffic for connections whose server/client ports fall in the configured range.

Validation evidence and limitation

Measured: the six provided sample packets passed the simulator check (6/6). Not measured: the assignment's hidden 20-packet set was not available in the local evidence, so no claim is made that those hidden cases were passed.

The subnet and NACL definitions are stored in network/subnets.md and network/nacl.json. The packet examples are in network/packets_sample.jsonl; the simulator is tools/nacl_sim.py.

Task 6 — Reproducibility, testing, CI/CD and security

The repository contains a GitHub Actions workflow with lint, unit-test, model-evaluation, quality-gate and secret-scan jobs. The documentation commit 524e961 (“Document Task 6 reproducibility and evidence”) was pushed to the fix/task-6-ci-docs branch by GitHub Copilot and fast-forwarded into the local main branch during final integration.

CI evidence

Evidence	Observed result / interpretation
Successful CI on documentation commit	Copilot reported CI run #9 with all five jobs passing: lint, unit-tests, evaluate-model, quality-gate, secret-scan. The run uploaded score and SARIF artifacts.
Earlier successful baseline run	GitHub Actions run 42526703693; this predates the documentation commit and is historical baseline evidence.
Deliberate quality-gate failure	Run 42526718020; untrained-model fixture produced JSON validity 0.0% against the 95% threshold, as intended. This is a demonstration failure, not a passing run.
Gitleaks	Secret-scan job passed in the reported CI run. Gitleaks is a secret-detection check, not a complete security audit.
Local checks after integration	Ruff passed; Python compile passed; pytest 7/7 passed; git diff --check passed.

Repository and release

Repository: https://github.com/alphacodes19/Cloud_Computing. Model release: https://github.com/alphacodes19/Cloud_Computing/releases/tag/model-v1. Release assets include v1/v2 compressed model archives and SHA256SUMS. Real Kubernetes credentials are excluded from version control; the local secret file is generated from k8s/secret.example.yaml and must remain untracked.

Fresh-clone reproducibility status

TASK6_REPRODUCIBILITY.md and README.md describe environment pins, artifact checksum verification, model evaluation, local cluster setup, and CI. However, a full fresh-clone `make up`/`make down` cycle was not demonstrated in the evidence available for this report. The deployment script has local prerequisites, including Kubernetes credentials and object-storage setup; these must be configured before reproducing the live service. The final documentation commit is integrated into the local main branch, but the output provided for this report did not show it pushed to origin/main or a final tag created.

AI-use disclosure

AI_USE.md documents AI assistance for repository inspection and Task 6 documentation. The author should review and complete the disclosure for all other tasks based on actual tools and prompts used. The model metrics in this report are taken from recorded result files rather than generated by AI.

Task 7 — Performance, cost and project review

Load-test measurements

Test	Requests / failures	p50 / p95	Throughput	Failure rate
Two-replica steady load, 2 min	319 total; 3 failed; 316 successful	1.3 s / 3.1 s	2.63 req/s (successful)	0.94%
Rolling update + rollback, 4 min	596 total; 19 failed	1.3 s / 3.3 s	2.49 req/s	3.19%
Pod deletion recovery, 2 min	270 total; 3 failed; 267 successful	1.7 s / 3.5 s	2.28 req/s	1.11%
Sequential API test	50 total; 1 failed	0.523 s / 0.977 s	Sequential	2.00%

Cost estimate (hypothetical rate card)

For two replicas, each limited to 1 vCPU and 1 GiB RAM, the assumed hourly compute cost is 2 × (INR 3.50 per vCPU-hour + INR 0.45 per GiB-hour) = **INR 7.90/hour**. Using the separate steady-load test's successful throughput of 316 / 120 = 2.633 successful requests/second gives approximately 9,480 successful requests/hour. Estimated cost per 1,000 successful requests = INR7.90 × 1,000 / 9,480 = **INR 0.83**. This is a simple compute-only estimate using the supplied hypothetical rate card; it excludes storage, networking, idle capacity variation, and real-provider charges.

SLO and error budget (test-window analysis)

For the four-minute rolling test, use a target of at least 99.5% HTTP 200 availability (maximum 0.5% failures). With 596 requests, the 0.5% budget is 2.98 request-equivalents; 19 failures were observed (3.19%), so the test-window budget was exceeded. For the two-minute Pod-recovery test, use the assignment's 2% failure ceiling: 270 × 2% = 5.4 request-equivalents; 3 failures (1.11%) stayed within that test budget. These are short test-window evaluations, not a claim of a measured monthly production SLO.

Six-area review

Area	Evidence-based review
Data / storage	2,864 cleaned records; SHA-256 recorded; 16 model objects verified in object storage.
Model quality	v2: 98% JSON validity, 81% intent accuracy, 37% full-frame exact match; student remains below teacher on full frame.
Latency / throughput	Steady load p95 3.1 s, 2.63 successful req/s; sequential p95 0.977 s.
Resource utilisation	During steady-load stress, CPU reached about 1,001m/999m and memory 735/701 MiB; no restarts in that test.
Reliability	Recovery: 1.11% failures (within 2%); rolling update: 3.19% (above 0.5%).
Security / cost	Gitleaks passed in recorded CI run; estimated compute cost INR 0.83 per 1,000 successful requests under the stated assumptions.

Conclusion and next steps

The project demonstrates the full workflow from teacher-labelled structured data through student fine-tuning, object storage, containerised inference, Kubernetes deployment, network policy simulation, CI evaluation and operational testing. The most important remaining technical gap is availability during rolling updates: the improved zero-unavailable strategy reduced disruption, but the observed failure rate still exceeded the rubric threshold. Further work should focus on keeping an old ready backend until a replacement is genuinely ready, isolating model-output errors from ingress errors, and validating the full workflow from a clean clone. The local single-laptop cluster and unavailable hidden packet set are additional evaluation limitations.

Primary evidence files: results/model_comparison.csv; results/task3-50-sequential-decode-fix.json; results/recovery_test_stats.csv and results/recovery_test_headers.json; results/final_rolling_stats.csv, results/final_rolling_failures.csv and results/final_rolling_headers.json; results/cold_start_three_trials.json; results/task1-cleaning-summary.txt; results/model-artifact-checksums.txt.